COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE31S1
DATE PERFORMED:	09 / 10 / 2026	**DATE SUBMITTED:**	
NAME:	Elijah I. Dizor	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1 Sorting Algorithms

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create C++ code for implementation of selection, bubble, insertion and merge sort

Create an array of elements with random values. The array must contain 100 elements that are not sorted. Use the created array for each sorting algorithm below. Show the output of the preparation task on table 7-1 in section 6.
Create a header file for the implementation of the different sorting algorithms.
Import this header file into your main.cpp.

A.1. Bubble Sort Technique

Code + Console Screenshot	

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

int main() {
    std::srand(static_cast<unsigned int>(std::time(nullptr)));

    const size_t array_size = 100;
    int data[array_size];

    for (size_t i = 0; i < array_size; i++) {
        data[i] = std::rand() % 1000;
    }

    std::cout <<"Unsorted Array\n\n";
    for (size_t i = 0; i < array_size; i++) {
        std::cout << data[i] << " ";
        if ((i + 1) % 10 == 0) std::cout << "\n";
    }
    std::cout << "\n";

    bubbleSort(data, array_size);

    std::cout <<"Bubble Sort\n\n";
    for (size_t i = 0; i < array_size; i++) {
        std::cout << data[i] << " ";
        if ((i + 1) % 10 == 0) std::cout << "\n";
    }
    std::cout << "\n";

    return 0;
}
```

```
#ifndef SORTING_H
#define SORTING_H
#include <iostream>
#include <algorithm>

template <typename T>
void bubbleSort(T arr[], size_t arrSize) {
    for (size_t i = 0; i < arrSize; i++) {
        for (size_t j = i + 1; j < arrSize; j++) {
            // Step 3: if A[J] > A[i]
            if (arr[j] > arr[i]) {

                std::swap(arr[j], arr[i]);
            }

        }

    }
}

#endif
```

Output:

```
Unsorted Array

499 593 804 491 871 215 297 228 440 344
95 672 101 164 956 315 701 546 724 322
113 156 36 199 402 640 77 405 636 952
699 761 660 467 263 586 718 237 809 331
723 653 538 638 88 966 521 379 122 448
140 497 633 32 586 924 538 605 671 168
346 180 511 573 143 111 582 436 369 569
216 264 889 557 220 352 297 104 918 124
117 460 574 590 872 190 755 404 426 658
203 747 624 261 319 366 244 756 723 814

Bubble Sort

966 956 952 924 918 889 872 871 814 809
804 761 756 755 747 724 723 723 718 701
699 672 671 660 658 653 640 638 636 633
624 605 593 590 586 586 582 574 573 569
557 546 538 538 521 511 499 497 491 467
460 448 440 436 426 405 404 402 379 369
366 352 346 344 331 322 319 315 297 297
264 263 261 244 237 228 220 216 215 203
199 190 180 168 164 156 143 140 124 122
117 113 111 104 101 95 88 77 36 32


--------------------------------
Process exited after 0.1836 seconds with return value 0
Press any key to continue . . .
```

Observations	The runs a nested loop to rearrange 100 random numbers generated in descending order. The main function creates an array that contains 100 elements. The outer loop (i) selects a starting position in the array, and the inner loop (j) starts one position ahead and scans through the remaining numbers

Table 7-1. Array of Values for Sort Algorithm Testing

A.2. Selection Sort Algorithm

Code + Console Screenshot	

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

void generateRandomArray(int arr[], size_t size) {
    for (size_t i = 0; i < size; i++) {
        arr[i] = std::rand() % 1000;
    }
}

void printArray(int arr[], size_t size) {
    for (size_t i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
        if ((i + 1) % 10 == 0) std::cout << "\n";
    }
    std::cout << "\n";
}

int main() {
    std::srand(static_cast<unsigned int>(std::time(nullptr)));
    const size_t array_size = 100;
    int data[array_size];

    generateRandomArray(data, array_size);
    std::cout << "Preparation Task: Unsorted Array\n\n";
    printArray(data, array_size);

    bubbleSort(data, array_size);
    std::cout <<"After Bubble Sort\n\n";
    printArray(data, array_size);

    generateRandomArray(data, array_size);
    std::cout <<"Preparation Task: Unsorted Array\n\n";
    printArray(data, array_size);

    selectionSort(data, static_cast<int>(array_size));
    std::cout <<"After Selection Sort\n\n";
    printArray(data, array_size);

    return 0;
}
```

```
#ifndef SORTING_H
#define SORTING_H

#include <iostream>
#include <algorithm>

template <typename T>
void bubbleSort(T arr[], size_t arrSize) {
    for (size_t i = 0; i < arrSize; i++) {
        for (size_t j = i + 1; j < arrSize; j++) {
            if (arr[j] > arr[i]) {
                std::swap(arr[j], arr[i]);
            }
        }
    }
}

template <typename T>
int Routine_Smallest(T A[], int K, const int arrSize);

template <typename T>
void selectionSort(T arr[], const int N) {
    int POS, temp, pass = 0;
    for (int i = 0; i < N - 1; i++) {
        POS = Routine_Smallest(arr, i, N);
        temp = arr[i];

        arr[i] = arr[POS];
        arr[POS] = temp;

        pass++;
    }
}

template <typename T>
int Routine_Smallest(T A[], int K, const int arrSize) {
    int position;
    T smallestElem = A[K];
    position = K;

    for (int j = K + 1; j < arrSize; j++) {
        if (A[j] < smallestElem) {
            smallestElem = A[j];
            position = j;
        }
    }
    return position;
}

#endif
```

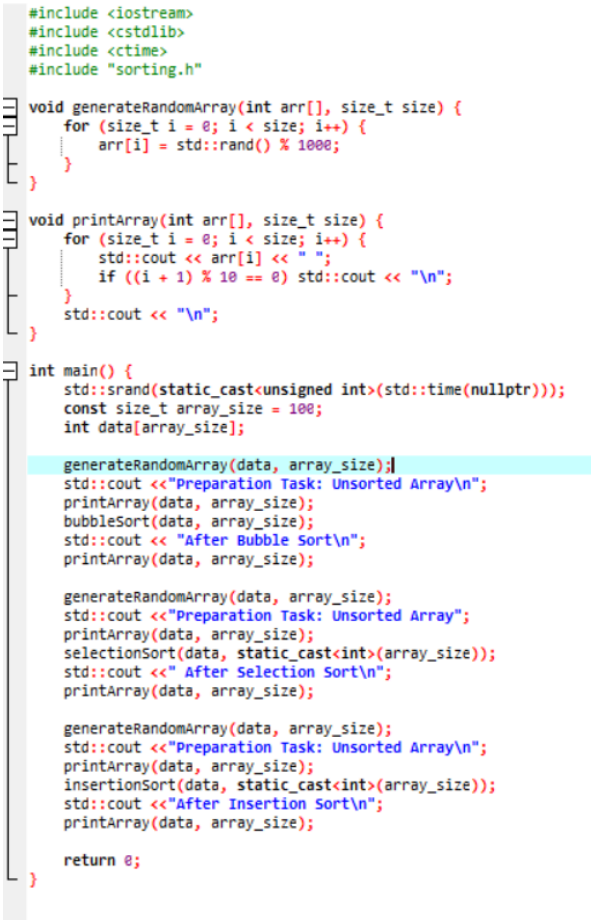
Output:

	Preparation Task: Unsorted Array 283 538 597 518 1 358 389 810 294 368 365 243 15 354 659 920 723 920 113 147 90 29 114 535 940 539 320 641 883 935 30 540 772 668 642 408 578 927 429 131 291 267 829 234 751 222 483 466 6 771 913 98 426 114 926 419 229 773 853 891 899 84 490 270 603 491 375 87 450 82 671 341 593 790 353 105 361 526 814 343 668 536 62 491 117 237 456 510 818 274 689 545 785 704 527 833 329 311 150 692 After Bubble Sort 940 935 927 926 920 920 913 899 891 883 853 833 829 818 814 810 790 785 773 772 771 751 723 704 692 689 671 668 668 659 642 641 603 597 593 578 545 540 539 538 536 535 527 526 518 510 491 491 490 483 466 456 450 429 426 419 408 389 375 368 365 361 358 354 353 343 341 329 320 311 294 291 283 274 270 267 243 237 234 229 222 150 147 131 117 114 114 113 105 98 90 87 84 82 62 30 29 15 6 1 Preparation Task: Unsorted Array 474 143 275 387 560 815 951 464 68 409 342 58 67 629 896 206 947 630 808 992 956 722 747 411 147 469 315 285 581 960 821 310 433 414 288 387 558 592 375 478 328 537 337 680 573 604 255 378 701 437 901 384 665 397 612 783 987 730 991 785 231 406 749 126 344 415 343 797 341 895 115 108 758 717 192 460 402 540 18 400 554 18 616 383 588 643 444 984 0 823 971 902 767 913 494 812 78 114 960 786 After Selection Sort 0 18 18 58 67 68 78 108 114 115 126 143 147 192 206 231 255 275 285 288 310 315 328 337 341 342 343 344 375 378 383 384 387 387 397 400 402 406 409 411 414 415 433 437 444 460 464 469 474 478 494 537 540 554 558 560 573 581 588 592 604 612 616 629 630 643 665 680 701 717 722 730 747 749 758 767 783 785 786 797 808 812 815 821 823 895 896 901 902 913
Observations	It scans an array of 100 random numbers generated to find the smallest value and systematically rearranges them in correct order from left to right. The outer loop variable i, keeps track of the current boundary position

	that needs to be filled with the next smallest number

Table 7-2. Bubble Sort Technique

A.3. Insertion Sort Algorithm

Code + Console Screenshot	

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

void generateRandomArray(int arr[], size_t size) {
    for (size_t i = 0; i < size; i++) {
        arr[i] = std::rand() % 1000;
    }
}

void printArray(int arr[], size_t size) {
    for (size_t i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
        if ((i + 1) % 10 == 0) std::cout << "\n";
    }
    std::cout << "\n";
}

int main() {
    std::srand(static_cast<unsigned int>(std::time(nullptr)));
    const size_t array_size = 100;
    int data[array_size];

    generateRandomArray(data, array_size);
    std::cout <<"Preparation Task: Unsorted Array\n";
    printArray(data, array_size);
    bubbleSort(data, array_size);
    std::cout << "After Bubble Sort\n";
    printArray(data, array_size);

    generateRandomArray(data, array_size);
    std::cout <<"Preparation Task: Unsorted Array";
    printArray(data, array_size);
    selectionSort(data, static_cast<int>(array_size));
    std::cout <<" After Selection Sort\n";
    printArray(data, array_size);

    generateRandomArray(data, array_size);
    std::cout <<"Preparation Task: Unsorted Array\n";
    printArray(data, array_size);
    insertionSort(data, static_cast<int>(array_size));
    std::cout <<"After Insertion Sort\n";
    printArray(data, array_size);

    return 0;
}
```

```cpp
#ifndef SORTING_H
#define SORTING_H
#include <iostream>
#include <algorithm>

template <typename T>
void bubbleSort(T arr[], size_t arrSize) {
    for (size_t i = 0; i < arrSize; i++) {
        for (size_t j = i + 1; j < arrSize; j++) {
            if (arr[j] > arr[i]) {
                std::swap(arr[j], arr[i]);
            }
        }
    }
}

template <typename T>
int Routine_Smallest(T A[], int K, const int arrSize);

template <typename T>
void selectionSort(T arr[], const int N) {
    int POS, pass = 0;
    T temp;
    for (int i = 0; i < N - 1; i++) {
        POS = Routine_Smallest(arr, i, N);
        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
        pass++;
    }
}

template <typename T>
int Routine_Smallest(T A[], int K, const int arrSize) {
    int position = K;
    T smallestElem = A[K];
    for (int j = K + 1; j < arrSize; j++) {
        if (A[j] < smallestElem) {
            smallestElem = A[j];
            position = j;
        }
    }
    return position;
}
```

```
template <typename T>
void insertionSort(T arr[], const int N) {
    int K = 1, J;
    T temp;

    while (K < N) {

        temp = arr[K];

        J = K - 1;

        while (J >= 0 && temp <= arr[J]) {

            arr[J + 1] = arr[J];

            J--;

        }
        arr[J + 1] = temp;
        K++;
    }

}

#endif
```

Output:

```
Preparation Task: Unsorted Array
263 796 664 552 181 729 852 364 803 955
529 689 774 516 404 236 961 695 272 428
101 22 578 762 920 605 125 379 116 839
809 339 778 151 250 300 710 290 878 496
637 35 270 748 964 310 608 805 170 577
437 407 569 408 351 980 977 41 813 778
342 39 866 816 946 42 867 785 302 455
624 322 358 388 844 337 74 111 952 711
740 55 789 752 463 795 601 487 76 218
180 735 43 603 403 936 336 46 68 847

After Bubble Sort
980 977 964 961 955 952 946 936 920 878
867 866 852 847 844 839 816 813 809 805
803 796 795 789 785 778 778 774 762 752
748 740 735 729 711 710 695 689 664 637
624 608 605 603 601 578 577 569 552 529
516 496 487 463 455 437 428 408 407 404
403 388 379 364 358 351 342 339 337 336
322 310 302 300 290 272 270 263 250 236
218 181 180 170 151 125 116 111 101 76
74 68 55 46 43 42 41 39 35 22
```

```
Preparation Task: Unsorted Array
164 953 635 865 14 64 836 825 889 991
411 288 392 770 441 316 567 95 855 357
427 26 520 91 197 401 761 396 979 561
989 661 352 809 959 710 582 21 486 755
504 622 392 665 236 40 856 670 209 516
146 147 542 31 946 326 787 675 255 219
604 596 421 348 467 182 379 697 501 741
618 313 328 740 112 457 82 866 852 484
494 98 778 980 251 323 529 95 342 191
932 150 97 679 742 435 188 527 417 921

 After Selection Sort
14 21 26 31 40 64 82 91 95 95
97 98 112 146 147 150 164 182 188 191
197 209 219 236 251 255 288 313 316 323
326 328 342 348 352 357 379 392 392 396
401 411 417 421 427 435 441 457 467 484
486 494 501 504 516 520 527 529 542 561
567 582 596 604 618 622 635 661 665 670
675 679 697 710 740 741 742 755 761 770
778 787 809 825 836 852 855 856 865 866
889 921 932 946 953 959 979 980 989 991
```

```
Preparation Task: Unsorted Array
603 614 176 552 544 449 219 304 130 537
899 838 373 482 487 461 620 132 623 391
237 87 343 725 276 555 982 555 435 902
95 38 649 18 991 706 416 918 169 640
811 483 54 453 343 906 817 529 409 980
234 918 624 330 545 271 885 646 990 51
745 962 406 962 93 544 17 183 520 145
81 919 484 890 261 993 906 907 512 70
0 530 491 483 968 720 562 372 362 594
38 604 197 427 614 626 322 231 20 726

After Insertion Sort
0 17 18 20 38 38 51 54 70 81
87 93 95 130 132 145 169 176 183 197
219 231 234 237 261 271 276 304 322 330
343 343 362 372 373 391 406 409 416 427
435 449 453 461 482 483 483 484 487 491
512 520 529 530 537 544 544 545 552 555
555 562 594 603 604 614 614 620 623 624
626 640 646 649 706 720 725 726 745 811
817 838 885 890 899 902 906 906 907 918
918 919 962 962 968 980 982 990 991 993
```

Observations	The main function calls 'generateRandomArray' to reset the data array with a third set of 100 random values that ranges from 0 to 999. The array is sent to 'insertionsort' as the function initializes a tracker variable k, assuming the very first item at index 0 is already a sorted sub-array

Table 7-3. Selection Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

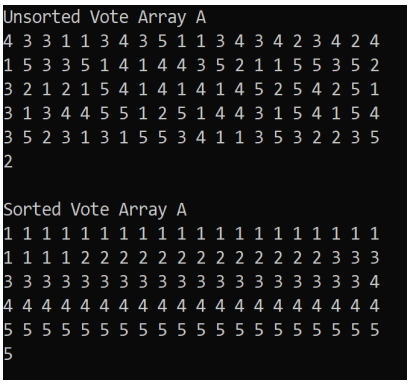
ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

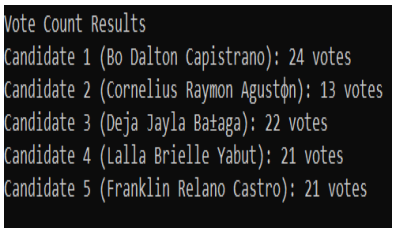
Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

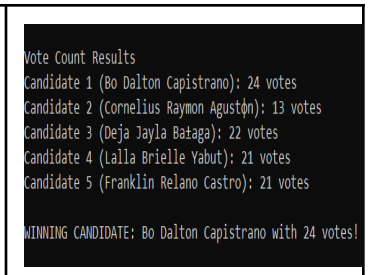
List of Candidates

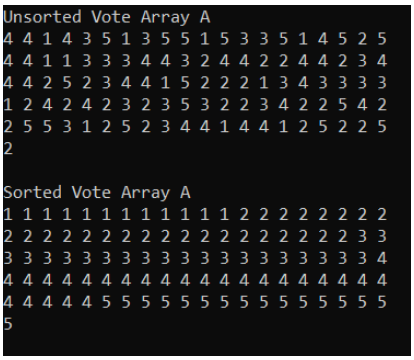
Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

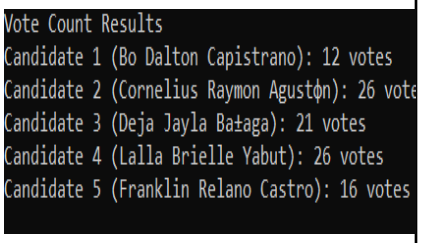
Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm

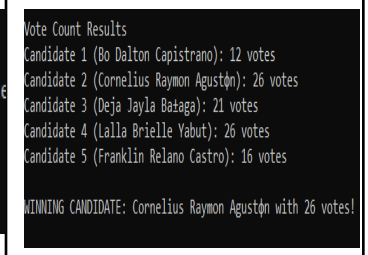


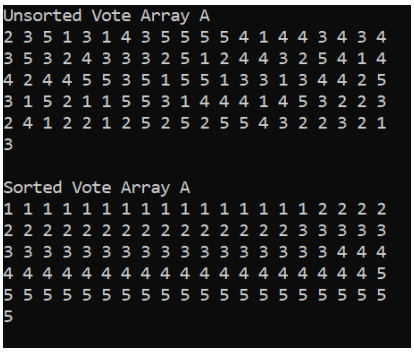


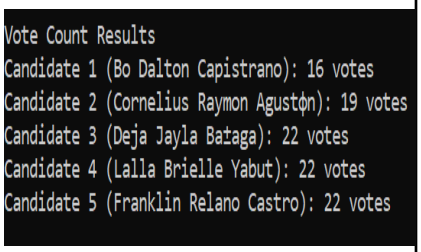


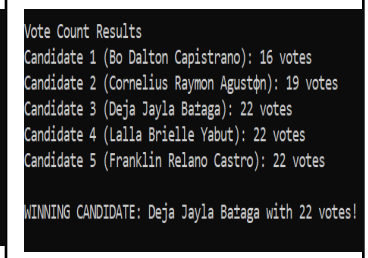












Based on the code, it runs an automated ballot processing system that utilizes an array based sorter to tally votes and determine the winner of the election. It initializes a static integer array with a size of 101 elements to represent submitted ballots. It then maps five candidate names to numeric IDs ranging from 1 to 5, using a for loop statement combined with a std::rand() % 5 + 1

Learning analysis

In completing the activity, I gained an understanding of how fundamental sorting algorithms operate. In all honesty, I was confused at first with the difference between the implementations of bubblesort, selection sort, and

insertionsort. But when i ran the code, I can now see the differences and their possible advantage and disadvantages based on real life scenario problems. Bubblesort, highlights the behaviour of adjacent comparisons, selectionsort demonstrates the efficiency of scanning the entire datasets and locate the absolute smallest element possible within the arrays, insertionsort has taught me the value of maintaining a dynamically growing sorted subarrays.

In completing the supplementary activity, i learned how to integrate sort and search algorithms to solve a practical data processing problem by applying insersionsort to the randomized ballots.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.